

Relatório do Trabalho 3: Sistema de Matchmaking

Maria Letícia
Cleidyene Renata

1. Algoritmos de ordenação implementados

Foram implementados manualmente dois algoritmos de ordenação: *insertion sort* e *merge sort*. Em ambos, o critério usado foi ordenar os jogadores primeiro por *score*, em ordem crescente, e, em caso de empate, por *timestamp*, também em ordem crescente. Essa escolha garante um comportamento determinístico: entre jogadores com o mesmo nível de habilidade, aparece primeiro aquele que chegou antes à fila.

No *insertion sort*, a estratégia foi manter uma parte inicial do array já ordenada. A cada iteração, um jogador é escolhido como chave e inserido na posição correta, deslocando para a direita os jogadores anteriores que deveriam vir depois dele. Essa solução foi escolhida por ser simples, usar apenas o próprio array e não exigir memória auxiliar relevante. Sua desvantagem é que, em entradas grandes ou muito desordenadas, muitos deslocamentos podem ser necessários.

No *merge sort*, foi usada a técnica de divisão e conquista. O array é dividido em duas partes, cada parte é ordenada separadamente e, ao final, as duas metades são intercaladas em ordem. Na etapa de intercalação, foram usados arrays auxiliares temporários para guardar as metades esquerda e direita. Essa escolha aumenta o uso de memória, mas reduz o custo de ordenação em entradas grandes.

2. Uso da recursão no *merge sort*

A recursão no *merge sort* foi usada para transformar o problema de ordenar o array inteiro em vários problemas menores. A função recursiva recebe um intervalo, indicado pelos índices inicial e final. Se o intervalo possui zero ou um elemento, ele já está ordenado e a recursão termina. Caso contrário, o intervalo é dividido em duas metades.

Depois da divisão, a função chama a si mesma para ordenar a metade esquerda e depois a metade direita. Quando as duas partes já estão ordenadas, a função de *merge* intercala os elementos, comparando *score* e *timestamp*, e copia o resultado de volta para o array principal. Como há aproximadamente $\log n$ níveis de divisão e cada nível realiza trabalho total linear na intercalação, o custo final é $O(n \log n)$.

3. Aproveitamento da ordenação para formar grupos

A ordenação foi aproveitada diretamente na formação dos grupos. Como os jogadores ficam ordenados por *score*, jogadores com habilidades próximas tendem a ficar próximos no array. Assim, o método não precisa testar todas as combinações possíveis de jogadores; ele avalia apenas blocos consecutivos de tamanho `groupSize`, seguindo a ordem atual do array.

Para cada bloco candidato, basta comparar o primeiro e o último jogador. Como o array está ordenado, o primeiro possui o menor *score* do bloco e o último possui o maior. Portanto, o grupo é válido quando

$$\text{maior_score} - \text{menor_score} \leq \Delta.$$

Essa decisão simplifica a verificação, pois evita comparar todos os pares de jogadores dentro do grupo.

O método percorre o array da esquerda para a direita e retorna o primeiro grupo válido encontrado, conforme a regra determinística do enunciado. Quando um grupo é formado, seus jogadores são copiados para um novo array, retornados pela função e removidos da fila por deslocamento dos elementos restantes. Caso nenhum bloco válido seja encontrado, nenhum jogador é removido.

4. Custo computacional das principais operações

Considerando n como o número de jogadores na fila e g como o tamanho do grupo solicitado, a inserção tem custo $O(1)$, pois o jogador é colocado diretamente no final do array. A remoção por ID tem custo $O(n)$ no pior caso, já que pode exigir uma busca linear e, após encontrar o jogador, o deslocamento dos elementos seguintes. A recuperação dos jogadores também custa $O(n)$, pois é necessário copiar todos os elementos para um novo array.

O *insertion sort* possui melhor caso $O(n)$, quando o array já está ordenado, mas caso médio e pior caso $O(n^2)$. O *merge sort* possui custo $O(n \log n)$, pois divide o problema recursivamente e intercala os elementos em tempo linear por nível. Em espaço auxiliar, o *insertion sort* usa $O(1)$, enquanto o *merge sort* usa $O(n)$ por causa dos arrays temporários.

Operação	Tempo	Espaço auxiliar
Inserção	$O(1)$	$O(1)$
Remoção por ID	$O(n)$	$O(1)$
Ordenação com <i>insertion sort</i>	$O(n^2)$	$O(1)$
Ordenação com <i>merge sort</i>	$O(n \log n)$	$O(n)$
Formação de grupo, já ordenado	$O(n)$	$O(g)$
Recuperação dos jogadores	$O(n)$	$O(n)$

5. Comparação experimental entre *insertion sort* e *merge sort*

Os testes experimentais foram feitos com diferentes quantidades de jogadores. Para tornar a comparação justa, os mesmos jogadores foram inseridos em dois objetos **Matchmaking**: um foi ordenado com *insertion sort* e o outro com *merge sort*. Os *scores* foram gerados por uma função pseudoaleatória determinística, e os tempos foram medidos com a biblioteca `<chrono>`. Cada valor da tabela representa a média de três execuções, em milissegundos, considerando apenas o tempo da ordenação.

Jogadores	Insertion sort (ms)	Merge sort (ms)
100	0.0321	0.0455
500	0.8084	0.1741
1000	2.4682	0.3566
3000	23.4794	1.0983
5000	56.8939	2.0438
10000	217.7557	4.1921

Os resultados seguem o comportamento esperado pela análise assintótica. Para entradas pequenas, a diferença entre os algoritmos é pequena. Porém, conforme n aumenta, o *insertion sort* cresce rapidamente, refletindo seu custo quadrático. O *merge sort*, por outro lado, cresce de forma mais moderada, compatível com $O(n \log n)$. Assim, o *insertion sort* é aceitável para poucos jogadores ou listas quase ordenadas, enquanto o *merge sort* é a escolha mais adequada para filas grandes.